

20 Python Interview Questions, Free Sample

Python

Every question was generated from real code, run for real, and checked against the actual output before being kept. Every wrong answer is explained too, not just the right one.

How this book was built

Most interview question banks are written by hand and quietly carry the mistakes of whoever wrote them. This one was built the other way around. Every code snippet in this book was run for real, and the printed answer key is the printed output, not someone's memory of what it should be. Where two independent ways of running the same snippet disagreed, the question was dropped rather than guessed at.

Each question also explains why the three wrong answers are wrong, not only why the right one is right. That is the part most practice tests skip, and it is the part that actually teaches you something the second time you get a question wrong.

Python Questions

Block 1, Data Types, Numbers and Strings

1. What does this print?

```
a = 'hello'
b = 'hello'
print(a is b)
```

- A. False
- B. True
- C. None
- D. 1

2. What does this print?

```
s = 'banana'
print(s.count('an'))
```

- A. 6
- B. 2
- C. 1
- D. 3

Block 2, Scope, Functions and Closures

3. What does this print?

```
def counter():
    n = 0
    def tick():
        n += 1
        return n
    return tick
t = counter()
print(t())
```

- A. 0
- B. UnboundLocalError
- C. 1
- D. NameError

4. What does this print?

```
def build(value, acc=[]):
    acc.append(value)
    return acc
a = build(1)
b = build(2, [])
c = build(3)
print(a, c)
```

- A. [1, 3] [1, 3]
- B. [1, 2, 3] [1, 2, 3]
- C. [1] [3]
- D. [1, 3] [3]

5. What does this print?

```
def clear_it(container):  
    container.clear()  
data = {'a': 1, 'b': 2}  
clear_it(data)  
print(data)
```

- A. {'a': 1}
- B. {}
- C. {'a': 1, 'b': 2}
- D. None

6. What does this print?

```
def greet(name: str) -> str:  
    return 'hello ' + name  
print(greet(5) if False else greet('5'))
```

- A. hello 5
- B. hello '5'
- C. TypeError
- D. hello 5.0

Block 3, Classes, Objects and Special Methods

7. What does this print?

```
class Lazy:  
    def __getattr__(self, name):  
        return f'missing:{name}'  
  
l = Lazy()  
l.x = 1  
print(l.x, l.y)
```

- A. 1 1
- B. 1 missing:y
- C. AttributeError
- D. missing:x missing:y

8. What does this print?

```
class Config:
    def __init__(self, **kwargs):
        self._data = kwargs

    def __getattr__(self, name):
        try:
            return self._data[name]
        except KeyError:
            raise AttributeError(name)

c = Config(host='localhost', port=8080)
print(c.host, c.port)
```

- A. None None
- B. AttributeError
- C. localhost 80
- D. localhost 8080

Block 4, Lists, Tuples and Slicing

9. What does this print?

```
a = [3, 1, 2]
b = a.sort()
print(b)
```

- A. None
- B. [1, 2, 3]
- C. [3, 1, 2]
- D. True

10. What does this print?

```
a = [[1, 2], [3, 4]]
b = a.copy()
b[0].append(99)
print(a)
```

- A. [[1, 2, 99], [3, 4], [1, 2, 99], [3, 4]]
- B. [[1, 2], [3, 4]]
- C. [[1, 2, 99], [3, 4]]
- D. [[1, 2], [3, 4, 99]]

11. What does this print?

```
print(any([]))
```

- A. False
- B. ValueError
- C. True
- D. None

12. What does this print?

```
print(all([1, 2, 0]))
```

- A. None
- B. False
- C. True
- D. 0

Block 5, Dictionaries, Sets and Comprehensions

13. What does this print?

```
a = {1, 2, 3}
b = {3, 4, 5}
print(sorted(a | b))
```

- A. [1, 2, 3, 3, 4, 5]
- B. [3]
- C. [1, 2, 3, 4, 5]
- D. [1, 2]

14. What does this print?

```
a = {1, 2, 3}
b = {2, 3, 4}
print(sorted(b - a))
```

- A. [2, 3, 4]
- B. [4]
- C. [1, 2, 3]
- D. [1]

Block 6, Iterators, Generators and Decorators

15. What does this print?

```
from functools import partial

def greet(greeting, name):
    return f'{greeting}, {name}'

hello = partial(greet, 'Hello')
print(hello('Sam'))
```

- A. Sam, Hello
- B. Hello, Hello
- C. greeting, Sam
- D. Hello, Sam

16. What does this print?

```
def counted(func):
    def wrapper(*args, **kwargs):
        wrapper.calls += 1
        return func(*args, **kwargs)
    wrapper.calls = 0
    return wrapper

@counted
def ping():
    return 'pong'

ping()
ping()
ping()
print(ping.calls)
```

- A. 0
- B. 1
- C. 3
- D. 4

Block 7, Mutability, References and Copying

17. What does this print?

```
def add_item(lst):
    lst.append('x')

items = ['a', 'b']
add_item(items)
print(items)
```

- A. ['a', 'b']
- B. ['a', 'b', 'x']
- C. ['x']
- D. ['a', 'b', 'x', 'x']

18. What does this print?

```
a = [1, 2]
b = a
a = a + [3]
print(b)
```

- A. [3]
- B. []
- C. [1, 2]
- D. [1, 2, 3]

19. What does this print?

```
def grow(lst):  
    lst = lst + [99]  
  
data = [1, 2]  
grow(data)  
print(data)
```

- A. [1, 2, 99]
- B. [1, 2]
- C. [99]
- D. []

20. What does this print?

```
import copy  
  
original = {'items': [1, 2]}  
clone = copy.copy(original)  
clone['items'] = [9, 9]  
print(original['items'])
```

- A. []
- B. [1, 2, 9, 9]
- C. [1, 2]
- D. [9, 9]

Answers and Explanations

Organized in the same order as the questions. Each option is explained, the correct one and the three distractors.

Python Answers

1. Correct answer, **B**

A. False

Assumes only numeric literals benefit from caching, and that string literals are always built as separate objects even when identical.

B. (correct) True

Correct. Short string literals that look like identifiers are automatically interned by CPython, so both names end up pointing to the same object.

C. None

Confuses a successful identity check with a failed lookup that would produce a missing value.

D. 1

Reports the truthy integer one instead of the actual boolean literal that is produced.

CPython automatically interns string literals that look like valid identifiers, reusing a single object for every occurrence of the same text within the same compiled code. This is an implementation detail, the language itself never promises that equal strings share the same object.

2. Correct answer, **B**

A. 6

Counts individual matching letters instead of occurrences of the whole two letter substring.

B. (correct) 2

Correct. count scans the string from left to right and counts every non overlapping occurrence of the substring.

C. 1

Stops after finding the first occurrence instead of continuing to scan the rest of the string.

D. 3

Counts overlapping occurrences of the pattern as if each shifted starting position counted separately.

The count method finds every non overlapping occurrence of a substring, moving its search position past the end of each match before looking for the next one. It never counts a match that shares characters with a match it already found.

3. Correct answer, **B**

A. 0

Assumes the read of n inside tick falls back silently to the enclosing scope while the increment is simply dropped.

B. (correct) UnboundLocalError

Correct. Without a nonlocal declaration, the assignment `n += 1` makes n local to tick, so the read of n on the right hand side happens before that local has any value, raising UnboundLocalError.

C. 1

Assumes tick can freely mutate the enclosing n with += the same way nonlocal would allow, without declaring it.

D. NameError

Confuses the local variable that is known but unassigned with a name that the interpreter has never seen anywhere in the function.

Augmented assignment like `n += 1` both reads and writes `n`, and the write alone is enough to mark `n` as a local variable for the whole function. Without a `nonlocal` statement pointing at the enclosing counter's `n`, `tick`'s own local `n` has never been assigned when the read happens, which raises `UnboundLocalError` instead of quietly falling back to the enclosing scope.

4. Correct answer, A

A. (correct) [1, 3] [1, 3]

Correct. Calls `a` and `c` both omit `acc` and therefore share the single default list, ending up with 1 and 3 appended to the same object, while `b` explicitly passes its own fresh empty list that is never touched by the others.

B. [1, 2, 3] [1, 2, 3]

Assumes the explicit empty list passed for `b` is the same shared default object, so its append also lands in the shared list.

C. [1] [3]

Assumes every call to `build`, including the one that omits `acc`, receives its own separate empty list.

D. [1, 3] [3]

Assumes `a` and `c` end up pointing at different list objects even though both calls used the same default argument.

Only calls that fall back to the default argument share the single object created at definition time. A call that explicitly passes its own list, like `build(2, [])`, receives a brand new object that is completely independent of the shared default, so it never mixes with the values accumulated through the default.

5. Correct answer, B

A. {'a': 1}

Assumes `clear` only removes one arbitrary entry from the dictionary rather than removing every key it contains.

B. (correct) {}

Correct. `container` refers to the same dictionary object as `data`, so calling `clear` on it empties that shared dictionary of every key it held, which is why `data` prints as an empty dictionary afterward.

C. {'a': 1, 'b': 2}

Assumes the function operates on a private copy of the dictionary, so calling `clear` on it never affects the caller's own dictionary.

D. None

Confuses the fact that `clear_it` has no return value with the idea that `data` itself gets replaced by that same `None`.

Methods like `clear` that mutate a mutable object in place affect every name that refers to that same object, since there is only ever one underlying object being changed. This is the same sharing behavior that makes `append` on a shared list or a key assignment on a shared dictionary visible through any reference to that object.

6. Correct answer, A

A. (correct) hello 5

Correct. Since the condition is `False`, only `greet('5')` actually runs, and because annotations are never enforced, the string `'5'` is concatenated directly onto `'hello '`, producing `'hello 5'`.

B. hello '5'

Assumes the string argument keeps its quote characters when concatenated, as if it were the code representation of the string rather than its plain text content.

C. TypeError

Assumes the str annotation is enforced and that passing a value which does not match it raises an error before the function body runs.

D. hello 5.0

Assumes the string argument is somehow converted to a float representation before being concatenated.

A conditional expression only evaluates the branch selected by its condition, the other branch's function call never executes at all. As with any other annotation, the str hint on name has no effect on which argument types are accepted, the function simply concatenates whatever value it receives.

7. Correct answer, B

A. 1 1

Assumes the fallback method also somehow affects the value of x, making both reads return the same number. x was assigned the value 1 directly and is read back unchanged, y is the only name missing from the instance.

B. (correct) 1 missing:y

Correct. `__getattr__` only runs for attributes that normal lookup fails to find, so the explicitly set x is returned directly and only y triggers the fallback.

C. AttributeError

Assumes accessing an attribute that does not exist on the instance always raises, even when `__getattr__` is defined. Defining `__getattr__` specifically replaces the usual `AttributeError` with whatever the method decides to return.

D. missing:x missing:y

Assumes `__getattr__` intercepts every attribute access, even ones that were explicitly set. `__getattr__` only fires when normal lookup fails to find the attribute, x was set directly on the instance so it is found without ever calling `__getattr__`.

`__getattr__` is only called as a fallback, after Python has already checked the instance dictionary and the class hierarchy and found nothing. Attributes that already exist, like an explicitly assigned x, are returned by normal lookup and never reach `__getattr__` at all.

8. Correct answer, D

A. None None

Assumes the fallback returns nothing usable for keys that were never set as real attributes. Both host and port were passed as keyword arguments and stored in `_data`, so the fallback finds and returns each of them.

B. AttributeError

Assumes accessing keys that only live inside the internal dictionary always fails with an error. `__getattr__` specifically catches the missing attribute case and redirects it to a dictionary lookup, returning the stored value instead of failing, since both keys exist.

C. localhost 80

Truncates the port number as if only part of it were stored or printed. The full integer 8080 is stored in the dictionary and returned exactly as it was passed in.

D. (correct) localhost 8080

Correct. host and port are not real attributes, so each access triggers `__getattr__`, which looks them up in the stored dictionary.

Storing arbitrary keyword arguments in a dictionary and then defining `__getattr__` to look names up in that dictionary is a common way to expose dynamic attributes without declaring them individually. Since `__getattr__` only runs for names missing from the instance, and host and port were never set directly, both accesses are redirected into the stored dictionary.

9. Correct answer, A

A. (correct) None

Correct. `list.sort()` sorts the list in place and returns `None`, so `b` ends up holding `None` regardless of how `a` itself changed.

B. [1, 2, 3]

Assumes `sort()` returns the sorted list, the way `sorted()` does, instead of mutating in place and returning nothing.

C. [3, 1, 2]

Assumes `sort()` returns a reference to the list before it was sorted.

D. True

Assumes an in place mutating method reports success as a boolean rather than returning `None`.

Methods that mutate a list in place, such as `sort`, `reverse`, `append` and `extend`, consistently return `None` so that their result is never mistaken for a new object. To get the sorted list itself, the value has to be read from the variable that was sorted, not from the method call.

10. Correct answer, C

A. [[1, 2, 99], [3, 4], [1, 2, 99], [3, 4]]

Assumes copying the outer list duplicates its inner lists as well, doubling the total number of sublists.

B. [[1, 2], [3, 4]]

Assumes `copy()` makes a fully independent copy at every level, so mutating a shared inner list through `b` would not be visible through `a`.

C. (correct) [[1, 2, 99], [3, 4]]

Correct. `copy()` only makes a shallow copy, so `b` holds new outer list slots that still point at the very same inner lists as `a`, and mutating one of those inner lists through `b` is visible through `a` as well.

D. [[1, 2], [3, 4, 99]]

Assumes the `append` lands on the second inner list instead of the first, misreading which sublist index 0 refers to.

A shallow copy duplicates only the outer container, the references it holds still point at the same nested objects as the original. To get inner lists that can be mutated independently, a deep copy is needed instead of a shallow one.

11. Correct answer, A

A. (correct) False

Correct. `any()` reports whether at least one element of the iterable is truthy, and an empty iterable has no elements at all, so the answer is `False`.

B. ValueError

Assumes `any()` requires at least one element to operate on and raises when given an empty iterable.

C. True

Confuses `any()` with `all()`, assuming an empty iterable vacuously satisfies the condition `any()` is checking for.

D. None

Assumes an empty iterable produces no boolean result at all instead of a definite `False`.

`any()` and `all()` are defined so that they always work on empty iterables without raising an error, they just disagree on what an empty iterable implies. `any()` finds no truthy element to point to, so it returns `False`, while `all()` has no falsy element to point to, so it returns `True`.

12. Correct answer, B

A. None

Assumes encountering a falsy element makes `all()` report that there is no valid answer instead of a definite False.

B. (correct) False

Correct. `all()` requires every element to be truthy, and the 0 in this list is falsy, so the overall result is False even though the other two elements are truthy.

C. True

Checks only that the list itself is non empty instead of checking the truthiness of each individual element.

D. 0

Assumes `all()` returns the first falsy element it finds instead of a plain boolean.

`all()` short circuits as soon as it finds a single falsy element and returns False at that point, regardless of how many truthy elements came before or after it. A single 0 anywhere in the iterable is enough to make the whole result False.

13. Correct answer, C

A. [1, 2, 3, 3, 4, 5]

Assumes the shared element 3 appears twice in the union because it exists in both sets. A set can never hold a duplicate value, an element that belongs to both operands still appears only once in the union.

B. [3]

Assumes `|` computes the intersection of the two sets instead of their union. The `|` operator is set union, it combines every element from both sets, the shared element 3 is not the only thing kept.

C. (correct) [1, 2, 3, 4, 5]

Correct. The union operator combines every distinct element from both sets, giving all of 1 through 5 once each, since 3 is shared it still only appears a single time.

D. [1, 2]

Assumes union keeps only the elements unique to the first set, similar to a difference operation. Union keeps every element present in either set, elements from b, like 4 and 5, are included too.

The `|` operator on two sets returns their union, a new set containing every element that appears in either operand, with duplicates naturally collapsed since a set cannot hold the same value twice. `sorted()` is used here only to get a deterministic printable order, it does not change which elements are present.

14. Correct answer, B

A. [2, 3, 4]

Assumes the minus operator removes nothing here since the sets overlap heavily. `b - a` still actively removes any element of b that is also present in a, 2 and 3 are shared and get removed, only 4, which is unique to b, remains.

B. (correct) [4]

Correct. `b - a` keeps only the elements of b that are not present in a, 2 and 3 are shared with a and get removed, leaving 4 as the only element unique to b.

C. [1, 2, 3]

Assumes the difference of two sets keeps everything except the truly identical set, essentially returning b unchanged. Set difference removes exactly the elements common to both sets, since a and b share 2 and 3, those two elements are dropped from the result.

D. [1]

Confuses this difference with `a - b` computed on the same two sets, which would remove different elements. `b - a` keeps elements of b missing from a, that is 4 alone, computing it the other way around is what would give 1 as the only leftover element.

Set difference is order sensitive, `b - a` and `a - b` answer different questions, one asks what is in b but not a, the other asks what is in a but not b. Here only the number 4 belongs to b and not to a, so it is the sole survivor.

15. Correct answer, D

A. Sam, Hello

Swaps the order of the fixed argument and the one supplied at call time.

B. Hello, Hello

Assumes the argument supplied at call time is ignored and the fixed value fills both slots.

C. greeting, Sam

Assumes the parameter name itself, rather than its fixed value, appears in the output.

D. (correct) Hello, Sam

Correct. `partial` fixes the first positional argument to 'Hello', so the argument supplied at call time fills the remaining name slot.

partial fixes the leading positional argument to Hello, so any further positional argument passed to hello fills the next open slot, which is name. The order of the fixed and remaining arguments is preserved exactly as declared in the original function signature.

16. Correct answer, C

A. 0

Assumes the counter attribute never actually gets incremented.

B. 1

Assumes only the first call updates the counter.

C. (correct) 3

Correct. Each call to `ping` increments the counter stored on the wrapper function object, and it was called three times.

D. 4

Counts one extra call that never actually happened.

Each call to ping actually invokes wrapper, which increments an attribute stored directly on the function object before delegating to the real implementation. Because that counter lives on the wrapper itself rather than being reset per call, it accumulates across all three invocations.

17. Correct answer, B

A. ['a', 'b']

Assumes the function receives a private copy of the list, so the caller would not see the appended value.

B. (correct) ['a', 'b', 'x']

Correct. The parameter `lst` is bound to the same list object as `items`, so the `append` call inside the function mutates the caller's list directly.

C. ['x']

Assumes `lst` starts as its own empty list rather than aliasing the caller's list.

D. ['a', 'b', 'x', 'x']

Assumes the `append` is somehow applied twice, once inside the function and once after it returns.

Python passes arguments by assigning the caller object to the parameter name, it never copies the argument. A mutable object like a list can therefore be changed from inside a function and the caller will observe the change, because the parameter and the caller variable are two names for one object.

18. Correct answer, C

A. [3]

Assumes `b` ends up bound to the freshly built list instead of keeping its original reference.

B. []

Assumes the plain assignment on a clears the object that b points to.

C. (correct) [1, 2]

Correct. The plus operator builds a brand new list and only a is rebound to it, b still refers to the original, unchanged list.

D. [1, 2, 3]

Confuses this with the augmented case and assumes the addition mutates the original list that b also references.

Unlike +=, the plain + operator on two lists never mutates either operand, it always returns a new list holding the concatenated elements. Rebinding a to that new list has no effect on any other name that still points to the list a used to reference, which is exactly the situation for b.

19. Correct answer, B

A. [1, 2, 99]

Confuses this with the augmented case, assuming the caller sees the extra element as well.

B. (correct) [1, 2]

Correct. `lst + [99]` builds a new list and the assignment only rebinds the local name `lst`, it never touches the list object that `data` still refers to.

C. [99]

Assumes only the freshly built list survives and the caller ends up holding it instead of its original list.

D. []

Assumes rebinding a parameter inside a function clears the caller variable.

Unlike +=, the plain + operator never mutates its operands, it always produces a new list. Assigning that new list to the local name `lst` only changes what `lst` refers to inside the function, the caller variable `data` was never touched and keeps pointing at the original two element list after the function returns.

20. Correct answer, C

A. []

Assumes reassigning a key on the copy clears the matching entry in original.

B. [1, 2, 9, 9]

Assumes the new list is appended to the existing one instead of replacing it inside clone's own mapping.

C. (correct) [1, 2]

Correct. Reassigning the key on the shallow copy only changes what clone maps that key to, it never mutates the list object that original still holds under the same key.

D. [9, 9]

Confuses reassigning a key on the copy with mutating the shared list value in place.

A shallow copy of a dictionary is still its own independent mapping, even though the values it holds are shared with the source. Rebinding a key on the copy, as opposed to mutating the value stored under it, only changes the copy's own mapping and leaves the original dictionary's entry exactly as it was.

That was 20 of the 600 Python questions.

The full bank is one PDF laid out exactly like this sample, all 600 questions first and the complete answer key after, with a reason for each of the four options. Every snippet in it was run the same way before its answer was written.

Get it for 9 euros at <https://ko-fi.com/s/00207c81fe>, or all three languages in one PDF for 19 euros at <https://ko-fi.com/s/fa83c4ed59>.

The same 20 questions can be played in a browser at <https://thibaudlepan77-svg.github.io/interview-questions-verified/>. If an answer looks wrong to you, open an issue on the repository linked from that page with the question number and what your own run printed.